VulnScout: An Always-Current, Tool-Grounded Assistant for Vulnerability Triage

Yuanhao Cao
MS in Information Systems
Northeastern University
cao.yuanh@northeastern.edu



Figure 1: What VulnScout is meant to produce, using real data. For "Log4j 2.14.1", the NVD API returns 10 CVEs that affect this version; we ranked them with the CISA Known Exploited Vulnerabilities (KEV) catalog and FIRST EPSS exploit probabilities, all queried on 2026-10-27. Six of the ten were published in 2025–2026, after many LLMs' training cutoffs, and ranking by CVSS alone would place three low-probability 7.5-score CVEs above two older CVEs that have EPSS above 0.97.

Abstract

Thousands of software vulnerabilities are disclosed every week, and teams without dedicated security staff struggle to answer two questions: *does this vulnerability affect the exact version I run*, and *what should I fix first?* General-purpose LLMs answer from a frozen training snapshot and often guess version ranges. We propose VulnScout, a conversational assistant that takes a natural-language description of a software stack, retrieves current vulnerability records from public sources (NVD, CVE records, OSV), decides affected versions with a deterministic, provenance-tracked resolver instead of the LLM, ranks findings by real-world exploitation evidence (CISA KEV, EPSS) rather than severity alone, and returns cited explanations and remediation advice. The target users are developers and IT operators who need fast, trustworthy triage.

1. Introduction

1.1. Target market

Vulnerability management is a core operational need for every organization that runs software, and its volume keeps growing. In the seven days before 2026-10-27 alone, the National Vulnerability Database (NVD) published 4,433 new CVE records [14], more than 600 per day. Severity scores do not settle what to patch first: CVSS describes how bad a vulnerability *could* be, while only a small fraction of vulnerabilities is ever exploited. This gap motivated data-driven signals such as the Exploit Prediction Scoring System (EPSS) [9, 10] and CISA's Known Exploited Vulnerabilities (KEV) catalog [5], which listed 1,760 vulnerabilities

on the same date.

The data itself is also becoming harder to use. In a sample of 2,000 CVEs published in that week, only 23.0% carried NVD's machine-readable list of affected products and versions (CPE configurations), and 52% were still "Awaiting Analysis". The vendor-submitted CVE records [4] usually do state affected versions (all 40 records in a random sample of the remaining CVEs did), but in inconsistent formats, for example a single string "26.7.0-26.7.1" instead of structured bounds.

1.2. Target users

VulnScout targets developers, DevOps engineers and IT operators in small and mid-sized organizations, and students running their own services, who have no dedicated security team. Their typical questions are informal, such as "our Java service still runs Log4j 2.14.1, what should we fix first?", and they need answers that are (1) *current*, covering vulnerabilities disclosed this week; (2) *version-exact*, stating whether their specific version is affected; (3) *prioritized* by real-world risk; and (4) *verifiable*, with links to official records.

1.3. Problem

General-purpose LLMs fail all four requirements in characteristic ways. Their knowledge stops at a training cutoff, while for Log4j 2.14.1 alone six of the ten affecting CVEs were published in 2025–2026 (Figure 1). Their answers about version ranges are generated rather than computed: the NVD record of Log4Shell (CVE-2021-44228) lists 396 affected-product entries, each with its own version bounds and exclusions. They tend to rank by severity, and they can fabricate plausible identifiers, a failure mode already documented for software packages [17]. Existing scanners solve version matching for structured inputs but do not offer conversational, explained triage (Section 2).

2. Related Work

Vulnerability data and prioritization. NVD [14] enriches CVE records with CVSS scores, weakness types (CWE) and CPE configurations; the CVE Program publishes the original CNA-submitted records in CVE JSON 5 [4]; OSV [7] provides package-ecosystem version ranges for open-source software. For prioritization, EPSS [9] estimates the probability that a vulnerability will be exploited in the next 30 days, and its later versions use community data to improve over severity-based strategies, which are poor predictors of exploitation [10]. CISA KEV [5] records vulnerabilities with confirmed exploitation. VulnScout builds on these sources rather than replacing them.

Software composition analysis (SCA). Scanners such as Trivy [3], Grype [2], OSV-Scanner [8] and Dependabot [6] match the dependencies in manifests, lock files or container images against vulnerability databases. They are mature and accurate for structured inputs, but they expect those inputs, return lists rather than answers to questions, and are not designed for free-form descriptions ("the Cisco router in our office") or follow-up questions. VulnScout is complementary: it can use the same databases, while adding natural-language understanding, explanation and dialogue.

LLMs in cybersecurity. A systematic review by Xu *et al*. [21] surveys LLM applications across security tasks. CyberMetric [19] evaluates general cybersecurity knowledge with multiple-choice questions, and CTIBench [1] evaluates cyber-threat-intelligence tasks motivated by concerns about reliability and hallucination. These benchmarks measure what a model *knows*; VulnScout instead keeps facts outside the model. Open-source LLM prototypes over NVD exist [15, 13], but we did not find one that evaluates version correctness or freshness systematically.

Retrieval, tool use and freshness. Retrieval-augmented generation (RAG) [12] conditions generation on retrieved documents, and ReAct [22] and Toolformer [16] let models call external tools during reasoning. FreshQA [20] and RealTime QA [11] show that static LLMs fail on fast-changing knowledge and that search augmentation helps. VulnScout applies these ideas to a domain where freshness is measurable (publication dates) and where some decisions, such as version comparison, should be computed rather than generated.

3. Motivation and Technical Contribution

Motivation. The facts that matter in vulnerability triage, such as affected versions, exploitation status and scores, change daily and are available from authoritative APIs, while the hard parts for a user are interpreting a messy description, reconciling inconsistent sources and explaining the result. We therefore let the LLM do language work and let deterministic tools do lookups and comparisons.

Technical contributions. The first two are the core of the project; the third and fourth complete the system.

1. **Multi-source, provenance-tracked version resolution.** Affected versions are resolved in order of reliability: NVD CPE configurations, CNA-submitted CVE records [4], OSV package ranges [7], and finally the description text. The LLM only *normalizes* heterogeneous version strings into structured bounds; a deterministic comparator decides whether the user's version is affected. Each verdict records its source and a confidence level, which addresses the 77% of new CVEs that NVD has not yet enriched.

2. **Exploitation-aware prioritization.** Findings are ranked by confirmed exploitation (KEV, including ransomware use), then exploitation probability (EPSS), then severity (CVSS), and the assistant explains the

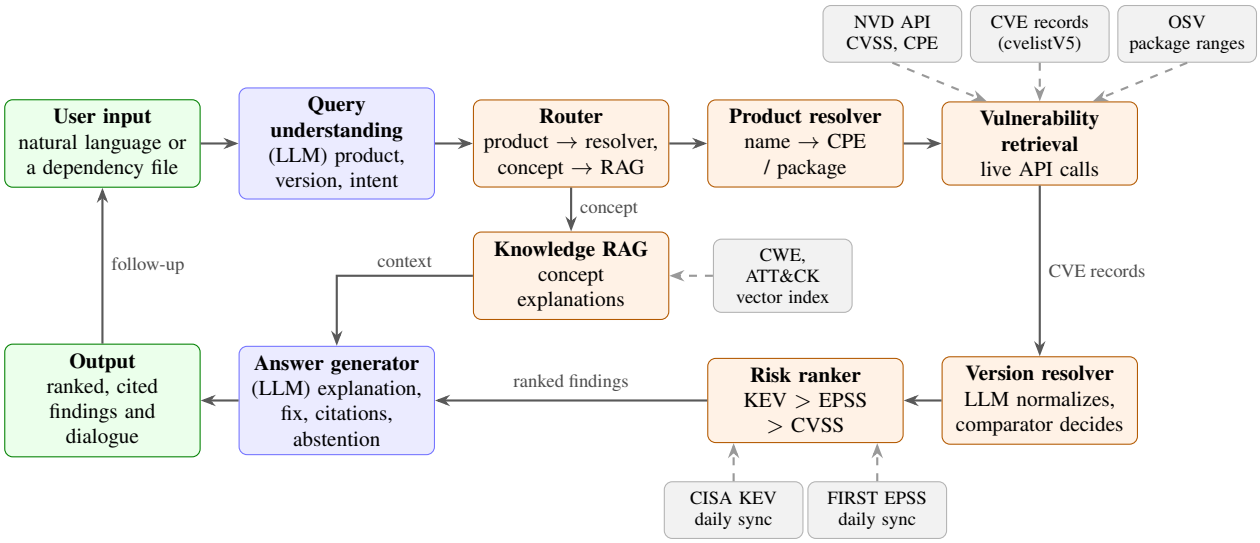


Figure 2: Preliminary system design. Blue components use the LLM; orange components are deterministic tools; gray boxes are external data sources (dashed arrows). The LLM interprets the question, normalizes version strings and writes the answer, while product resolution, version comparison and ranking are computed. Conceptual questions bypass the vulnerability tools and use a local RAG index.

reason for each rank. As Figure 1 shows, this ordering differs substantially from sorting by CVSS.

3. **Always-current retrieval with routing.** Questions about specific products and versions trigger live API calls; conceptual questions (*e.g.*, "what is JNDI injection?") are answered from a local RAG index of CWE and ATT&CK [18] descriptions. The KEV catalog and EPSS scores are re-synchronized daily.

4. **Grounded answers with abstention.** Every factual claim cites an NVD, CVE, KEV or EPSS record, and the assistant explicitly says when a product cannot be identified or a version cannot be resolved, instead of guessing.

4. Preliminary System Design

Figure 2 shows the planned workflow.

- **Query understanding (LLM)** extracts the products, versions and intent from the user's message or parses a dependency file (*e.g.*, `pom.xml`, `requirements.txt`).
- **Router** sends product-specific questions to the vulnerability tools and conceptual questions to the knowledge RAG.
- **Product resolver** maps names such as "log4j" to CPE identifiers or OSV package names, and asks the user to choose when the name is ambiguous.
- **Vulnerability retrieval** queries the NVD API live and fetches the matching CVE and OSV records.
- **Version resolver** implements contribution 1 and outputs, for each CVE, *affected / not affected / unknown* with its source.
- **Risk ranker** implements contribution 2 using the daily KEV and EPSS snapshots.
- **Answer generator (LLM)** writes the explanation and remediation advice, using only the structured findings and retrieved text, with citations.

We plan to build the tools in Python with an LLM agent loop in the ReAct style [22], serve an open-weight 7–8B instruction-tuned model in fp16 on a V100 GPU of Northeastern's Explorer cluster, use a commercial API model as a reference, and expose a web chat interface (*e.g.*, Gradio).

References

[1] Md Tanvirul Alam, Dipkamal Bhusal, Le Nguyen, and Nidhi Rastogi. CTIBench: A benchmark for evaluating LLMs in cyber threat intelligence. *arXiv preprint arXiv:2406.07599*, 2024. 2

[2] Anchore. Grype. https://github.com/anchore/grype. Accessed 2026-10-27. 2

[3] Aqua Security. Trivy. https://github.com/aquasecurity/trivy. Accessed 2026-10-27. 2

[4] CVE Program. cvelistV5: CVE records in CVE JSON 5 format. https://github.com/CVEProject/cvelistV5. Accessed 2026-10-27. 2

[5] Cybersecurity and Infrastructure Security Agency. Known exploited vulnerabilities catalog. https://www.cisa.gov/known-exploited-vulnerabilities-catalog. Catalog version 2026.10.27, 1,760 entries; accessed 2026-10-27. 1, 2

[6] GitHub. Dependabot. https://github.com/dependabot/dependabot-core. Accessed 2026-10-27. 2

[7] Google. OSV: Open source vulnerability database. https://osv.dev. Accessed 2026-10-27. 2

[8] Google. OSV-Scanner. https://github.com/google/osv-scanner. Accessed 2026-10-27. 2

[9] Jay Jacobs, Sasha Romanosky, Benjamin Edwards, Michael Roytman, and Idris Adjerid. Exploit prediction scoring system (EPSS). *arXiv preprint arXiv:1908.04856*, 2019. Presented at Black Hat 2019. 1, 2

[10] Jay Jacobs, Sasha Romanosky, Octavian Suciu, Benjamin Edwards, and Armin Sarabi. Enhancing vulnerability prioritization: Data-driven exploit predictions with community-driven insights. *arXiv preprint arXiv:2302.14172*, 2023. 1, 2

[11] Jungo Kasai, Keisuke Sakaguchi, Yoichi Takahashi, Ronan Le Bras, et al. RealTime QA: What's the answer right now? *arXiv preprint arXiv:2207.13332*, 2022. 2

[12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2005.11401. 2

[13] morpheuslord. CVE-llm_dataset. https://github.com/morpheuslord/CVE-llm_dataset. Accessed 2026-10-20. 2

[14] National Institute of Standards and Technology. National vulnerability database (NVD) CVE API 2.0. https://services.nvd.nist.gov/rest/json/cves/2.0. Accessed 2026-10-27. 1, 2

[15] Kalyan Patel. NVD-Agentic-Chatbot. https://github.com/kalyan-patel/NVD-Agentic-Chatbot. Accessed 2026-10-20. 2

[16] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, et al. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023. 2

[17] Joseph Spracklen, Raveen Wijewickrama, A H M Nazmus Sakib, Anindya Maiti, et al. We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs. In *USENIX Security Symposium*, 2025. arXiv:2406.10279. 2

[18] The MITRE Corporation. MITRE ATT&CK. https://github.com/mitre/cti. Enterprise ATT&CK in STIX format; accessed 2026-10-27. 3

[19] Norbert Tihanyi, Mohamed Amine Ferrag, Ridhi Jain, Tamas Bisztray, and Merouane Debbah. CyberMetric: A benchmark dataset based on retrieval-augmented generation for evaluating LLMs in cybersecurity knowledge. *arXiv preprint arXiv:2402.07688*, 2024. 2

[20] Tu Vu, Mohit Iyyer, Xuezhi Wang, Noah Constant, et al. FreshLLMs: Refreshing large language models with search engine augmentation. *arXiv preprint arXiv:2310.03214*, 2023. 2

[21] Hanxiang Xu, Shenao Wang, Ningke Li, Kailong Wang, et al. Large language models for cyber security: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 2025. arXiv:2405.04760. 2

[22] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629. 2, 3